

安全漏洞审查报告

Flask 用户管理登录系统

项目名称：flask-user-login

审查日期：2026-07-07

审查工具：自动化安全审计 + 人工验证

风险等级：● 严重 2 项 | ● 高危 2 项 | ● 中危 5 项 | ● 低危 1 项

总计：10 项安全缺陷

机密文件

目录

一、漏洞汇总

二、漏洞详情

V-001 密码明文存储 严重

V-002 Secret Key 硬编码 高危

V-003 Debug 模式暴露调试器 严重

V-004 HTML 注释泄露默认账号 中危

V-005 密码明文展示在页面 中危

V-006 缺少暴力破解防护 高危

V-007 缺少 CSRF 保护 中危

V-008 Session Cookie 缺少安全标志 中危

V-009 无日志审计 低危

V-010 缺少 HTTPS 低危

三、修复优先级建议

四、修复代码示例

一、漏洞汇总

编号	漏洞名称	风险等级	CVSS
V-001	密码明文存储	严重	9.8
V-003	Debug 模式暴露调试器	严重	9.0
V-002	Secret Key 硬编码	高危	8.6
V-006	缺少暴力破解防护	高危	8.2
V-008	Session Cookie 缺少安全标志	中危	5.5
V-004	HTML 注释泄露默认账号	中危	5.3
V-005	密码明文展示在页面	中危	5.0
V-007	缺少 CSRF 保护	中危	5.0
V-010	缺少 HTTPS	低危	3.7
V-009	无日志审计	低危	3.5

二、漏洞详情

V-001 密码明文存储

严重

📁 app.py:14-27

🔒 不安全的数据存储

CVSS 9.8

密码以明文形式直接硬编码在源代码的 USERS 字典中：

```
USERS = {  
    "admin": {"password": "admin123", ...},  
    "alice": {"password": "alice2025", ...},  
}
```

⚠️ 风险：项目已上传至 GitHub 公开仓库，任何人可直接看到所有用户的明文密码。一旦泄露，攻击者可利用相同密码横向渗透用户在其他平台的账号（撞库攻击）。

✅ 修复：存储 SHA-256 哈希，登录时比对哈希值。

V-002 Secret Key 硬编码

高危

📁 app.py:6

🔒 加密凭证泄露

CVSS 8.6

```
app.secret_key = "dev-key-2025"
```

⚠️ 风险：攻击者可利用此弱密钥伪造任意用户的 session cookie，直接以 admin 身份登录。Flask 使用此密钥对 session 进行签名，密钥泄露 = session 可被完全伪造。

✅ 修复：从环境变量读取或使用 `os.urandom(32).hex()` 自动生成。

V-003 Debug 模式暴露调试器

严重

📁 app.py:72

🔖 配置不当

CVSS 9.0

```
app.run(debug=True, host="0.0.0.0", port=5000)
```

⚠️ 风险：Flask debug=True 会启用 Werkzeug 交互式调试器。当应用抛出异常时，攻击者可以在浏览器中直接打开 Python Shell，执行任意代码，完全控制服务器。监听 0.0.0.0 使局域网内任何设备均可访问。

✅ 修复：生产环境设置 debug=False，通过环境变量控制。

V-004 HTML 注释泄露默认账号

中危

📁 templates/login.html:1

🔖 敏感信息泄露

CVSS 5.3

```
<!-- 调试信息 - 默认管理员账号 用户名: admin 密码: admin123 -->
```

⚠️ 风险：浏览器查看页面源代码即可获得管理员的完整登录凭证，攻击者无需任何扫描即可获得有效账号。

✅ 修复：移除调试注释，或使用模板条件判断仅在开发环境显示。

V-005 密码明文展示在页面

中危

templates/index.html:9

🔑 敏感信息泄露

CVSS 5.0

```
<li><span>密码 :</span>{{ user.password }}</li>
```

⚠️ 风险：登录成功后用户的密码明文显示在页面上。旁观者（Shoulder surfing）、屏幕共享或截图时密码会被无意泄露。

✅ 修复：UI 上永远不要回显密码字段，或使用掩码显示。

V-006 缺少暴力破解防护

高危

app.py:39-47

🔑 认证安全

CVSS 8.2

```
if username in USERS and USERS[username]["password"] == password:
    session["username"] = username
```

⚠️ 风险：登录接口无任何失败次数限制。已知用户名（admin/alice），攻击者可用自动化脚本每秒尝试数百个密码。弱口令（admin123）可在几分钟内被爆破。

✅ 修复：实现基于 IP 的登录失败锁定机制（5 次失败后锁定 15 分钟）。

V-007 缺少 CSRF 保护

中危

app.py

🔑 跨站请求伪造

CVSS 5.0

登录和登出接口没有 CSRF token 保护。

⚠️ 风险：攻击者可构造恶意页面诱导用户访问，触发跨站请求（如强制登出）。

✅ 修复：使用 Flask-WTF 的 CSRFProtect 或在表单中添加随机 token。

V-008 Session Cookie 缺少安全标志

中危

📁 app.py

🔑 Cookie 安全

CVSS 5.5

Flask 默认 session cookie 未设置 HttpOnly、Secure、SameSite 属性。

⚠️ 风险：缺少 HttpOnly 导致 XSS 可窃取 session；缺少 Secure 导致明文传输；缺少 SameSite 导致跨站请求可携带 cookie。

✅ 修复：配置 `SESSION_COOKIE_HTTPONLY=True`、`SESSION_COOKIE_SAMESITE='Lax'`。

V-009 无日志审计

低危

📁 app.py

🔑 安全监控缺失

CVSS 3.5

⚠️ 风险：应用未记录任何登录日志。发生安全事件时无法溯源，无法判断攻击时间、来源 IP、攻击手法。

✅ 修复：使用 logging 模块记录成功/失败登录事件及来源 IP。

V-010 缺少 HTTPS

低危

📁 app.py

🔑 传输安全

CVSS 3.7

使用 RSA 加密了密码字段，但整体通信未使用 TLS 加密。

⚠️ 风险：虽然密码已加密，但攻击者可实施中间人攻击（MITM），替换服务器下发的公钥为自己控制的公钥，从而解密密码。Cookie、请求头等均为明文传输。

✅ 修复：配置 SSL 证书，使用 `ssl_context` 启动 HTTPS，或通过 Nginx 反向代理终止 TLS。

三、修复优先级建议

立即修复（P0）

漏洞	原因
V-001 明文密码	代码在 GitHub 公开，任何人可看到所有密码
V-003 Debug 模式	远程代码执行，服务器可被完全接管
V-002 Secret Key	Session 可被伪造，导致任意账号登录

尽快修复（P1）

漏洞	原因
V-006 暴力破解	admin 密码弱，几分钟即可被爆破
V-004 HTML 注释	已在 GitHub 公开，需更新代码

规划修复（P2）

漏洞	原因
V-005 密码展示	信息泄露 + 用户体验问题
V-007 CSRF	需用户交互，利用门槛较高
V-008 Cookie 安全	配合 HTTPS 效果最佳

长期跟进（P3）

漏洞	原因
V-009 日志审计	安全事件溯源时才有价值
V-010 HTTPS	可结合 Nginx 反向代理实现

四、修复代码示例

4.1 app.py (修复关键漏洞)

```
import os
import hashlib
import secrets
import logging
from datetime import datetime, timedelta
from collections import defaultdict
from flask import Flask, render_template, request, redirect, session, jsonify

app = Flask(__name__)
# 从环境变量读取 secret_key
app.secret_key = os.environ.get("FLASK_SECRET_KEY") or os.urandom(32).hex()

# Cookie 安全配置
app.config.update(
    SESSION_COOKIE_HTTPONLY=True,
    SESSION_COOKIE_SAMESITE='Lax',
)

# 日志配置
logging.basicConfig(filename='app.log', level=logging.INFO)

# 暴力破解防护
login_attempts = defaultdict(list)
MAX_ATTEMPTS = 5
LOCKOUT_MINUTES = 15

def is_locked_out(ip):
    attempts = login_attempts[ip]
    recent = [t for t in attempts if t > datetime.now() - timedelta(minutes=LOCKOUT_MINUTES)]
    login_attempts[ip] = recent
    return len(recent) >= MAX_ATTEMPTS

# 密码哈希存储 (仅示例, 实际应从数据库读取)
def hash_password(password, salt=None):
    if not salt:
        salt = secrets.token_hex(16)
```

```
        return hashlib.sha256((password + salt).encode()).hexdigest(), salt

# 启动时 debug 由环境变量控制
if __name__ == "__main__":
    debug_mode = os.environ.get("FLASK_ENV") == "development"
    app.run(debug=debug_mode, host="0.0.0.0", port=5000)
```

4.2 templates/login.html (移除调试注释)

```
{% if config.DEBUG %}
<!-- 调试信息 - 默认管理员账号 用户名: admin 密码: admin123 -->
{% endif %}
{% extends "base.html" %}
{% block content %}
...

```

4.3 templates/index.html (隐藏密码回显)

```
<!-- 移除密码字段展示 -->
<li><span>用户名:</span>{{ user.username }}</li>
<!-- <li><span>密码:</span>{{ user.password }}</li> -->
<li><span>邮箱:</span>{{ user.email }}</li>

```

— 报告结束 —